Project - High Level Design

On
CI/CD Pipeline For Retail App

Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Shubham Kumawat	EN22CS301943
2.	Shivanshu Verma	EN22CS301917
3.	Shivani Vishwakarma	EN22CS301913
4.	Shubh Kapadia	EN22CS301942
5.	Shobha Goswami	EN22CS301922

Group Name: 06 D8

Project Number: D0-41

Industry Mentor Name:

University Mentor Name: Avnesh Joshi

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Non-Functional Requirements
   6.1. Security Aspects
   6.2. Performance Aspects

7. References

1. Introduction

1.1. Scope of Document

The purpose of this document is to provide a high-level design for implementing a **CI/CD pipeline for a Node.js-based Retail Application**. It defines the architecture, components, workflows, and deployment strategies required to automate the software development and delivery lifecycle.

This document covers:

- Design of the **Continuous Integration (CI)** process, including code integration, automated build, and unit testing upon every code commit
- Design of the **Continuous Deployment (CD)** process, including Docker image creation, versioning, and automated deployment to the production environment
- Integration of tools such as **GitHub, Jenkins/GitHub Actions, Docker, AWS ECR, and AWS EC2**
- Description of system components, their interactions, and overall architecture
- High-level workflow from code commit to production deployment
- Security, scalability, and performance considerations for enterprise-level deployment
- API overview and application structure

This document does **not include low-level implementation details**, such as:

- Detailed source code
- Exact configuration scripts (e.g., Jenkinsfile, YAML files)
- Database schema or internal algorithm logic

The scope is limited to providing a **conceptual and architectural understanding** of the system to guide development, deployment, and future scalability.

1.2. Intended Audience

This document is intended for stakeholders involved in the design, development, deployment, and evaluation of the CI/CD pipeline for the Node.js Retail Application.

The primary audience includes:

- **Developers**
  To understand how code changes integrate into the CI/CD pipeline, build process, and deployment workflow.

- **DevOps Engineers**
  To design, implement, and maintain the CI/CD pipeline, infrastructure, and automation processes.
- **System Architects**
  To review the high-level architecture, component interactions, and scalability considerations.
- **Project Managers**
  To gain visibility into the development lifecycle, deployment strategy, and overall system flow.
- **Quality Assurance (QA) Engineers**
  To understand the automated testing process integrated within the CI pipeline.
- **University Mentors / Evaluators**
  To assess the project design, architecture quality, and adherence to industry best practices.

This document provides sufficient high-level information to help stakeholders understand the system without going into low-level implementation details.

1.3. System Overview

The system is designed to implement a **fully automated CI/CD pipeline** for a Node.js-based Retail Application. It enables continuous integration, testing, containerization, and deployment of the application in a reliable and scalable manner.

The architecture is divided into two primary layers:

1. Application Layer (Retail Application)

The application layer consists of the core business logic and user-facing components:

- Developed using **Node.js, Express, and TypeScript**
- Provides RESTful APIs for handling retail operations (e.g., product listing)
- Serves static frontend content from the `/public` directory or compiled `/dist` folder
- Includes a **health check endpoint (`/healthz`)** for monitoring application status
- Designed to be **stateless**, enabling easy scaling and containerization

2. DevOps Layer (CI/CD Infrastructure)

The DevOps layer manages the automation of build, test, and deployment processes:

- **GitHub**
  Used for version control and source code management
- **CI/CD Tool (Jenkins or GitHub Actions)**
  Automates:
  - Code build
  - Unit testing
  - Docker image creation
  - Deployment process
- **Docker**
  Packages the application into a container to ensure consistency across environments
- **AWS Elastic Container Registry (ECR)**
  Stores versioned Docker images securely
- **AWS EC2**
  Hosts the deployed application using Docker containers
- **Kubernetes**
  Enables container orchestration, auto-scaling, and high availability

3. System Workflow Summary

The high-level workflow of the system is as follows:

1. Developer pushes code to GitHub repository
2. CI/CD pipeline is automatically triggered
3. Application is built and tested
4. Docker image is created and tagged
5. Image is pushed to AWS ECR
6. Deployment is triggered on AWS EC2
7. EC2 pulls the latest image and runs the container
8. Application becomes accessible to end users

---

4. Key Features of the System

- Automated build and deployment pipeline
- Containerized application for portability
- Scalable and cloud-ready architecture
- Faster and more reliable release cycles
- Reduced manual intervention and errors

---

5. Benefits

- Improves development efficiency through automation
- Ensures consistent environments using Docker
- Enables quick rollback using image versioning

- Supports future scalability with Kubernetes

2. System Design

2.1. Application Design

The Retail Application follows a **modular, scalable, and container-ready architecture** using Node.js and Express.

Architecture Style

- Client-Server architecture
- RESTful API design
- Stateless application
- Containerized deployment

Application Structure

- `src/` → Core backend logic (Node.js + TypeScript)
- `public/` → Static frontend assets
- `dist/` → Production build output
- `Dockerfile` → Container configuration
- `ecosystem.config.cjs` → PM2 configuration (optional)
- `k8s/` → Kubernetes manifests (future scope)

Backend Design

- Built using Express framework
- Handles API requests and responses
- Lightweight and fast processing
- Reads data from JSON files

API Endpoints

- `GET /api/products` → Fetch product list
- `GET /healthz` → Health check

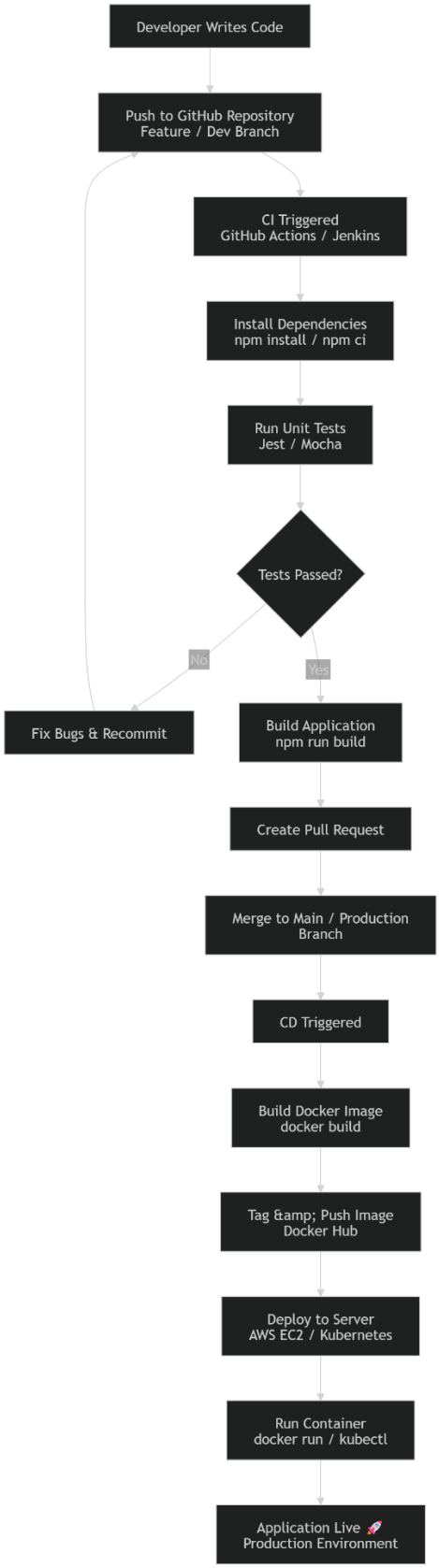
- Static UI served from `/public`
- Uses HTML, CSS, JavaScript
- Displays product data via APIs

Containerization

- Application packaged using Docker
- Ensures consistency across environments
- Runs on defined port

2.2. Process Flow

The CI/CD pipeline automates the process from code development to deployment.



Process flow Diagram

2.3. Information Flow

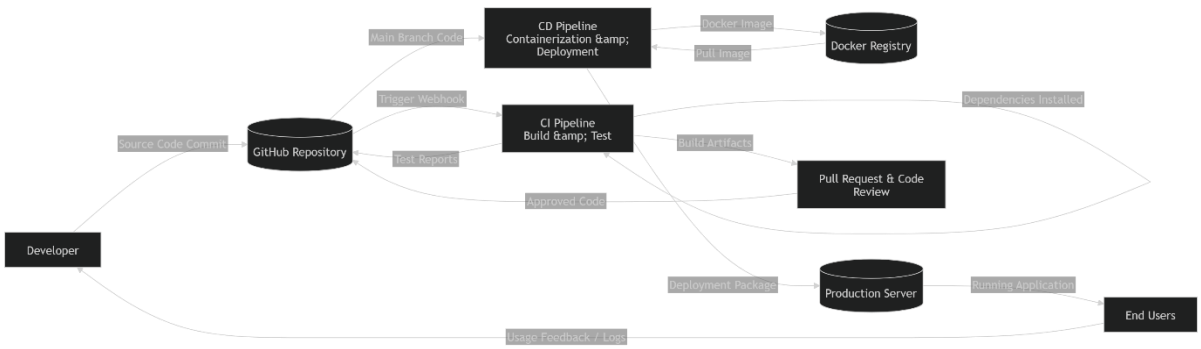
The system defines how data and requests move between different components:

User Interaction Flow

1. The user sends an HTTP request to the application hosted on **AWS EC2**
2. The EC2 instance forwards the request to the **Docker container**
3. The container runs the **Node.js application**
4. The application processes the request and retrieves data (from JSON/API)
5. The response is sent back through:
    - Application → Docker → EC2 → User

CI/CD Information Flow

1. Developer pushes code to **GitHub**
2. CI/CD pipeline is triggered
3. Code is built and tested
4. Docker image is created
5. Image is pushed to **AWS ECR**
6. EC2 pulls the latest image for deployment



Information Flow

2.4. Component Design

The system consists of multiple interacting components:

Component	Description
GitHub	Source code repository
CI/CD Tool (Jenkins/GitHub Actions)	Automates build, test, deploy
Node.js App	Backend service
Frontend	Static UI
Docker	Containerization platform
AWS EC2	Deployment server
Kubernetes	Future orchestration

2.5. Key Design Considerations

- **Automation**
  End-to-end CI/CD reduces manual intervention
- **Scalability**
  Stateless containers allow horizontal scaling
- **Reliability**
  Automated testing ensures code quality
- **Security**
  Credentials stored securely in CI/CD tools
- **Maintainability**
  Modular code structure
- **Portability**
  Docker ensures environment consistency
- **Rollback Strategy**
  Versioned Docker images allow easy rollback

2.6. API Catalogue

The Retail Application exposes a set of RESTful APIs to handle client requests and system monitoring. These APIs are lightweight, stateless, and designed for easy integration with frontend components and external systems.

---

1. Product API

- **Endpoint:** `/api/products`
- **Method:** `GET`

- **Description:**
  Retrieves the list of all available products from the backend data source (JSON file).
- **Request Parameters:**
  None
- **Response:**
  Returns a JSON array containing product details such as:
  - Product ID
  - Name
  - Price
  - Description

2. Health Check API

- **Endpoint:** `/healthz`
- **Method:** `GET`
- **Description:**
  Used to verify whether the application is running properly. This endpoint is commonly used by monitoring tools and CI/CD pipelines.
- **Request Parameters:**
  None
- **Response:**

API Design Characteristics

- RESTful architecture
- Stateless communication
- JSON-based responses
- Lightweight and fast
- Easy integration with frontend

3. Data Design

3.1. Data Model

The system uses a **simple JSON-based data model** instead of a traditional database.

Product Data Structure

Field	Data Type	Description
`id`	Integer	Unique product identifier
`name`	String	Name of the product
`price`	Number	Product price
`description`	String	Product details

4. Interfaces

The system provides interfaces for interaction between users, applications, and system components. These interfaces ensure smooth communication and data exchange within the system.

4.1 User Interface

The Retail Application provides a simple and user-friendly interface:

- Displays product listings
- Fetches data from backend APIs
- Built using **HTML, CSS, and JavaScript**
- Accessible through a web browser

Features

- Lightweight design
- Fast loading
- Easy navigation
- Responsive layout (basic)

4.2 External Interfaces

The system integrates with external services for deployment and automation:

- **GitHub** → Source code repository
- **CI/CD Tool (Jenkins / GitHub Actions)** → Automation pipeline
- **AWS EC2** → Hosting and deployment

4.53Interface Constraints

- Requires internet connectivity
- Dependent on AWS services availability
- Browser compatibility required for frontend

5. State and Session Management

The system is designed as a **stateless application**, meaning it does not store any user session or application state on the server side. Each request is processed independently, which improves scalability and simplifies deployment.

5.1 Application State

- The application does **not maintain any persistent state**
- All requests are handled independently
- Data is fetched from JSON files for each request
- No in-memory state is stored between requests

5.2 Session Management

- No session tracking or user authentication is implemented
- No session IDs or cookies are used
- Suitable for public, read-only applications

5.3 Benefits of Stateless Design

- **Scalability**
  Easily supports horizontal scaling (multiple containers)
- **Reliability**
  Failure of one instance does not affect others
- **Simplicity**
  No session storage or management overhead
- **CI/CD Friendly**
  Stateless services are easier to deploy and replace

6. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system such as performance, security, scalability, and reliability. These requirements ensure that the system operates efficiently and meets industry standards.

6.1 Reliability

- The system ensures **consistent and dependable operation**
- Automated CI/CD pipeline reduces human errors
- Docker containers provide stable runtime environments
- Failures in one container do not affect the entire system

6.2 Scalability

- The application is designed to be **horizontally scalable**
- Stateless architecture allows multiple instances to run simultaneously
- Future integration with **Kubernetes** enables auto-scaling
- AWS infrastructure supports vertical and horizontal scaling

6.3 Availability

- Application is hosted on **AWS EC2**, ensuring high availability
- System can run 24×7 with minimal downtime
- Health check API (`/healthz`) helps monitor application status
- Future load balancing can improve uptime further

6.4 Maintainability

- Modular code structure improves readability and maintenance
- CI/CD automation simplifies updates and deployments
- Version control using GitHub ensures easy tracking of changes
- Docker ensures consistency across environments

6.5 Usability

- Simple and user-friendly interface
- Easy navigation for product listing
- Lightweight frontend ensures fast interaction

6.6 Security Aspects

- Secure authentication using **SSH keys** for EC2 access
- AWS IAM roles for controlled access to ECR and resources
- Sensitive credentials stored securely in CI/CD tools
- No hardcoded secrets in source code

6.7 Performance Aspects

- Lightweight Node.js backend ensures fast request handling

- Docker containers start quickly (low startup time)
- Minimal resource consumption
- Future caching can further improve performance

6.8 Portability

- Docker ensures the application runs consistently across environments
- Can be deployed on any cloud platform or local system
- Reduces environment dependency issues

7. References

The following resources were referred to during the design and implementation of the CI/CD pipeline and the Node.js Retail Application:

Documentation

- **Node.js Official Documentation**
  https://nodejs.org/en/docs
- **Express.js Documentation**
  https://expressjs.com/
- **Docker Official Documentation**
  https://docs.docker.com/
- **GitHub Actions Documentation**
  https://docs.github.com/en/actions
- **Jenkins Documentation**
  https://www.jenkins.io/doc/

Cloud & DevOps Resources

- **AWS EC2 Documentation**
  https://docs.aws.amazon.com/ec2/

Concepts & Learning Resources

- CI/CD Pipeline Concepts (DevOps)
- Containerization using Docker
- Cloud Deployment using AWS
- REST API Design Principles